ANEXO 2 UT1: ENTRADA DE DATOS. CLASE SCANNER. (RA5)

1 CLASE SCANNER PARA LECTURA DE DATOS

La clase Scanner se encuentra disponible a partir de Java 5 y facilita la lectura de datos en los programas Java.

Primero veremos varios ejemplos de lectura de datos en Java con Scanner y después explicaremos en detalle cómo funciona.

Para utilizar Scanner en el programa tendremos que hacer lo siguiente:

1. Escribir el *import*
   La clase Scanner se encuentra en el paquete java.util, por lo tanto se debe incluir al inicio del programa la instrucción:

   ```
   import java.util.Scanner;
   ```

2. Crear un objeto Scanner
   Tenemos que crear un objeto de la clase Scanner asociado al dispositivo de entrada. Si el dispositivo de entrada es el teclado (lo más frecuente), escribiremos:

   ```
   Scanner sc = new Scanner(System.in);
   ```

   Se ha creado el objeto `sc` asociado al teclado representado por `System.in`.

3. Una vez hecho esto podemos leer datos introducidos por teclado. Para leer podemos usar el método nextXxx() donde Xxx indica el tipo, por ejemplo `nextInt()` para leer un entero, `nextDouble()` para leer un doublé, etc.

Ejemplos de lectura:

Ejemplo de lectura por teclado de un número entero:

```
int n;
System.out.println ("Introduzca un número entero: ");
n = sc.nextInt(); // asigna a la variable n el número int
                  // introducido por teclado
```

Ejemplo de lectura por teclado de un número tipo double:

```
double x;
System.out.print("Introduzca número de tipo double: ");
x = sc.nextDouble(); // asigna a la variable x el número double
                     // introducido por teclado
```

Ejemplo de lectura de una cadena de caracteres:

```
String s;
System.out.print("Introduzca texto: ");
s = sc.nextLine(); // asigna a la variable s el String introducido
                   // por teclado
```

Ejemplo de lectura de un número de tipo long:

```
long ln;
System.out.print("Introduzca un número: ");
ln = sc.nextLong(); // asigna a la variable ln el número long
                    // introducido por teclado
```

Ejemplo de lectura de un carácter:

```
char c;
System.out.print("Introduzca un carácter: ");
c = sc.next().charAt(0); // asigna a la variable c el primer carácter
                         // (carácter 0) de la cadena introducida por teclado
```

Ejemplo de programa con lectura de datos con Scanner:

El programa debe solicitar al usuario el nombre de la persona y mostrarlo por pantalla. Seguidamente lee por teclado el radio de una circunferencia de tipo *double* y muestra su longitud. Además, lee un entero y muestra el valor de elevarlo al cuadrado.

```
import java.util.Scanner;
public class Ejemplo1Scanner {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in); //crear un objeto Scanner
      String nombre;
      double radio;
      int n;

      System.out.print("Introduzca su nombre: ");
      nombre = sc.nextLine(); //leer un String
      System.out.println("Hola " + nombre + "!!!");

      System.out.print("Introduzca el radio de la circunferencia: ");
      radio = sc.nextDouble(); //leer un double
      System.out.println("Longitud de la circunferencia: " + 2
                         * Math.PI * radio);

      System.out.print("Introduzca un número entero: ");
      n = sc.nextInt(); //leer un entero
      System.out.println("El cuadrado es: " + Math.pow(n,2));
   }
}
```

2 Funcionamiento de la clase Scanner.

Es importante conocer el funcionamiento interno de la clase Scanner para poder realizar las operaciones de lectura de forma correcta. De forma resumida, el proceso de lectura por teclado en un programa Java es el siguiente:

- Los datos que se introducen desde teclado se almacenan en una zona de memoria que vamos a llamar *buffer*.

- Mediante un *stream* estos datos pasan al programa.

Un *stream* o flujo de datos es un objeto que hace de intermediario entre el programa y el origen o el destino de los datos. El programa lee del *stream* o escribe en él, sin importarle de donde proceden los datos físicamente o hacia qué dispositivo se dirigen realmente.

Un *stream* está formado por una secuencia de bytes utilizados para la entrada o salida de un programa.

Java crea de forma automática los siguientes *streams* cuando se ejecuta un programa:

- System.in: *stream* de entrada conectado al teclado
- System.out: *stream* de salida conectado al monitor
- System.err: *stream* de salida conectado al monitor para mensajes de error.



Además de estos *streams* estándar, Java proporciona una gran cantidad de clases para *streams* que permiten leer y escribir en ficheros.

Cuando en el programa aparece una instrucción de lectura de un dato por teclado, se accede al buffer de entrada en busca del dato. Si lo encuentra, lo extrae del buffer y lo incorpora al programa.

Si el dato no ha sido encontrado en el buffer, generalmente se deberá a que el buffer está vacío. Esta es la situación más habitual. En este caso el programa espera a que el usuario introduzca el dato por teclado. Cuando el usuario lo introduce y pulsa intro entonces se extrae el valor introducido del buffer y se incorpora al programa.

Si el dato encontrado no es del tipo esperado o de un tipo compatible se produce un error. Esto se produce, por ejemplo, cuando se está ejecutando el método nextInt() para extraer un entero del buffer y se introduce por ejemplo un *double* o un *char*. El tipo de dato encontrado en el buffer no coincide con el tipo que se quiere leer.

2.1 Cómo limpiar el buffer de entrada en Java.

Podemos limpiar el buffer mediante dos métodos:

next() extrae del buffer el siguiente *token* en forma de String. Debemos tener en cuenta que este método solo extrae un *token*, si hubiese más permanecerían en el buffer.

nextLine() extrae del buffer un *String* con todo el contenido del buffer hasta encontrar un salto de línea (\n), el cual se produce al pulsar Intro. El salto de línea no se incluye en el String devuelto. Este carácter \n se elimina del buffer.

Cuando se introducen datos de distinto tipo desde teclado habrá situaciones en la que será necesario eliminar totalmente el contenido del buffer de entrada. De otro modo la lectura de datos no se realizará de forma correcta, en particular cuando se intenten introducir datos de tipo *String*.

Ejemplo: Programa que lee por teclado el nombre, edad y dirección de una persona y lo muestra por pantalla.

```
import java.util.Scanner;
public class Ejemplo2Scanner {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      String nombre, direccion;
      int edad;

      System.out.print("Introduce tu nombre: ");
      nombre = sc.nextLine(); //leer el nombre
      System.out.print("Introduce tu edad: ");
      edad = sc.nextInt(); //leer la edad
      System.out.print("Introduce tu dirección: ");
      direccion = sc.nextLine(); //leer la dirección
      System.out.println("Datos introducidos");
      System.out.println("Nombre: " + nombre);
      System.out.println("Edad: " + edad);
      System.out.println("Dirección: " + direccion);
   }
}
```

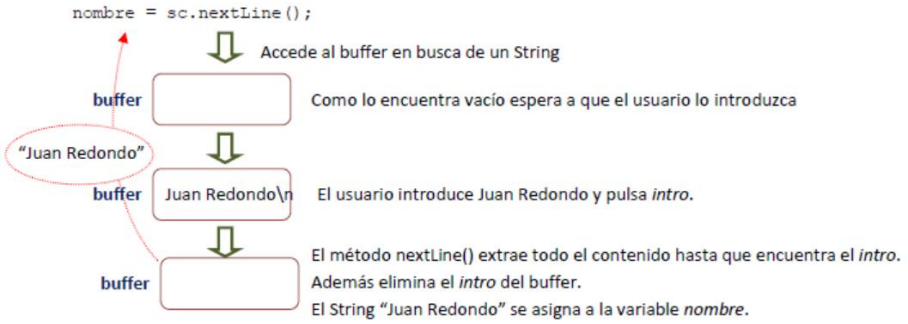
Si lo ejecutamos vemos que la lectura de datos no se realiza de forma correcta:

```
Introduce tu nombre: Juan Redondo
Introduce tu edad: 25
Introduce tu dirección: Datos introducidos
Nombre: Juan Redondo
Edad: 25
Dirección:
```

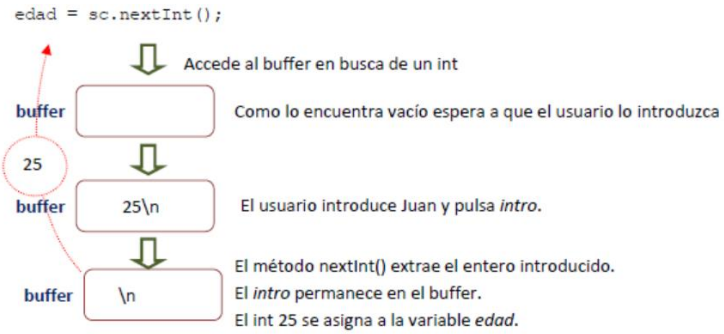
Se ha leído por teclado el nombre y la edad, pero el programa no se ha detenido para que el usuario introduzca la dirección por lo que la variable dirección queda sin valor.

La explicación a lo que ha ocurrido podemos verla de forma gráfica:

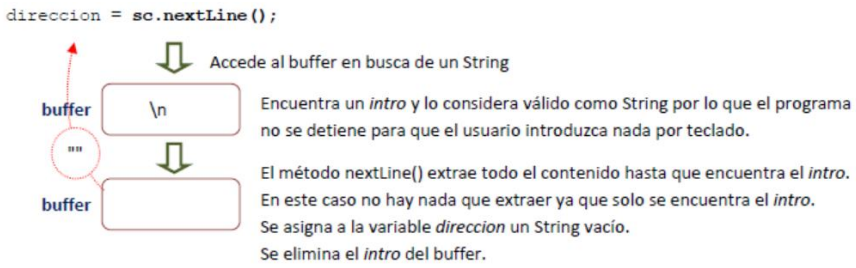
Primero se introduce el nombre por teclado:



La siguiente operación de lectura por teclado es la edad:



La siguiente operación de lectura por teclado es la dirección:



Después de leer la edad, el intro permanece en el buffer y si después intentamos leer un *String*, la lectura por teclado no se realizará.

La solución es sencilla: **Limpiar el buffer después de leer el número entero** mediante el método nextLine();

El programa quedaría así:

```
import java.util.Scanner;
public class Ejemplo2Scanner {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      String nombre, direccion;
      int edad;

      System.out.print("Introduce tu nombre: ");
      nombre = sc.nextLine(); //leer el nombre
      System.out.print("Introduce tu edad: ");
```

```
        edad = sc.nextInt(); //leer la edad

        sc.nextLine(); //limpiar el buffer de entrada

        System.out.print("Introduce tu dirección: ");
        direccion = sc.nextLine(); //leer la dirección
        System.out.println("Datos introducidos");
        System.out.println("Nombre: " + nombre);
        System.out.println("Edad: " + edad);
        System.out.println("Dirección: " + direccion);
    }
}
```

Ahora, después de introducir la edad, el método nextLine() elimina el intro que ha quedado en el buffer por lo que la lectura de la dirección se realizará de forma correcta.

```
Introduce tu nombre: Juan Redondo
Introduce tu edad: 25
Introduce tu dirección: C/Mayor 134 - Madrid.
Datos introducidos
Nombre: Juan Redondo
Edad: 25
Dirección: C/Mayor 134 - Madrid.
```

En general se debe **limpiar el buffer** de entrada de Scanner cuando tengamos que introducir un dato de tipo *String* después de haber introducido un dato de tipo numérico.